

What we're building — and how it becomes a Creator app

The programme, the data behind it, the rules you'll have to enforce yourself, and the order to build in. **No prior database knowledge assumed** — but by the end you'll know why the design looks the way it does.

~24

Creator forms

2

subforms

2

permission sets

23

screens designed

1

app, one codebase

Navigate with   or the buttons below.

We run a free training programme that takes students from college into jobs.

Colleges send candidates. We interview and pick the best. We train them for about a year. If they attend enough, they get certified. Then we help them get hired. **Every form you build serves some part of that.**

WHO SENDS STUDENTS

Colleges

Partner institutions. Classes run on their campus; they give us candidates.

WHO WE SERVE

Students

From families under ₹4 lakh a year. They pay a refundable deposit, nothing more.

WHO TEACHES

Trainers

Paid per hour taught. JavaScript, HTML, CSS, Zoho Creator and more.

WHO SELECTS

Panel members

External interviewers who score candidates. Also paid per hour.

WHO HIRES

Companies

Employers who interview our graduates and make offers.

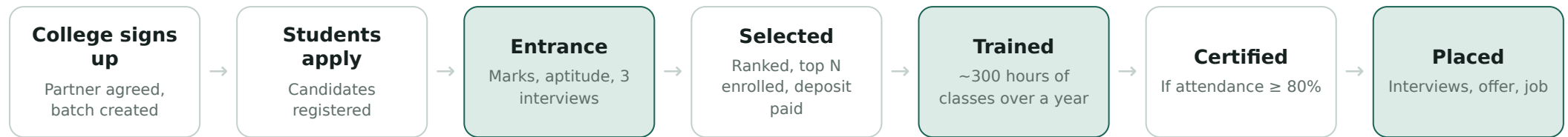
AND US

Coordinators

Staff who run all of it — and the main users of the app you're about to build.

One student's path through the system

Every form and report sits somewhere on this line. When you get lost in the detail, come back to it.



Running alongside the whole time: trainers and panel members record hours and get invoiced monthly. That's a second track through the same app, and it's where mistakes cost real money.

One app. Two very different logins.

Students, colleges and panel members get **no login at all** — this is an internal tool. Admins and trainers share the app but see almost nothing in common.

PERMISSION SET A · DESKTOP

Admin / Coordinator

Runs everything. 14 of the 23 screens.

- Colleges, batches, candidates
- Entrance scoring and selection
- Curriculum and timetable
- Certificates and billing

PERMISSION SET B · CREATOR MOBILE APP

Trainer

Used standing in a classroom. 5 screens.

- My schedule today
- Start / end a class
- Take attendance
- Enter test scores
- See my own invoices

PERMISSION SET C · DESKTOP

Placement officer

Only the job-hunting part. 3 screens.

- Companies and open roles
- Interview rounds
- Offers and salaries

Say this out loud now, because it shapes every report you build: a trainer must never load another trainer's sessions or invoices. In Creator that's a criteria filter on the report, not something you can bolt on later.

Every important noun becomes a form

That's genuinely the whole trick. Read the story back, underline the nouns, and each one gets a form with fields for the facts you need about it.

"St. Mary's College"	→	a record in Colleges
"The 2026-27 group"	→	a record in Batches
"Ananya, a candidate"	→	a record in Students
"Monday 9am JavaScript class"	→	a record in Class_Session
"Ananya was present"	→	a row in the Attendance subform
"June invoice for Priya"	→	a record in Billing

Notice the last three. **Events are nouns too** — "a class happened", "someone was present", "an invoice was raised". Beginners spot people and places, then miss the events. **Most of our forms are events.**

The spec is written in database words. Here's your translation.

The design doc says "table", "column", "foreign key". You'll be building forms and fields. Same ideas, different vocabulary — keep this slide handy while reading it.

Table — colleges	→	Form — Colleges
Row / record	→	Record — one submission
Column — college_name Text	→	Field — Single Line
Foreign key — batches.college_id	→	Lookup field pointing at Colleges
Numeric(10,2)	→	Currency or Decimal field
Text, One of (...)	→	Dropdown with those exact options
Boolean	→	Decision box
Timestamptz	→	Date-Time — or a free system field

Creator hands you four things free

The spec lists 28 tables. You will not build 28 forms — Creator already does some of this for you.

TWO WHOLE TABLES DISAPPEAR

users and roles

The spec designs its own login and permission tables. **Delete both.** Creator has users and permission sets built in — that's exactly what they were for.

FOUR COLUMNS ON EVERY FORM

System fields

The spec puts `id`, `created_at`, `updated_at` on all 28 tables. Creator gives you **ID, Added Time, Modified Time, Added User, Modified User** automatically. Don't recreate them.

THE "WHO DID THIS" COLUMNS

marked_by, entered_by, mapped_by

Audit columns recording who took the register or entered a score. Mostly just **Added User**. Only build a real field where it may differ from the person who saved the record.

THE TRAINER'S PHONE SCREENS

Creator mobile app

Five of our wireframes are phone-first. **You don't build a separate mobile app** — Creator renders your forms and reports on mobile already. Design those forms for a thumb.

Net effect: 28 tables → roughly 24 forms and 2 subforms. The next few slides explain which is which.

One-to-many: the "many" side holds the lookup

One college runs many batches. Where do you store the connection?

WRONG INSTINCT

Put three "batch" fields on the college

How many do you leave? Three? Ten? A college that runs an eleventh batch breaks the form. Nine empty fields on every other college.

RIGHT

Put one Lookup field on the batch

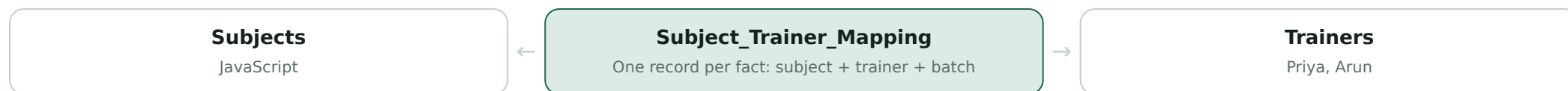
Each batch record points at its college. Unlimited batches, no empty fields, and the report writes itself — *show batches where College = this one*.

The rule: the "many" side stores the link. Each batch remembers its college; the college keeps no list. This same shape repeats everywhere — a batch has many students, a module has many topics, an invoice has many lines.

In Creator terms: a Lookup field on Batches pointing at Colleges. That single field is what the spec calls `batches.college_id`.

Many-to-many needs a form in the middle

Real example: **one subject is taught by several trainers, and one trainer teaches several subjects.** Now neither side can hold the lookup.



Each record in the middle form says one small thing: *"Priya teaches JavaScript in batch 2026-27."* Two trainers on a subject? Two records. Three? Three records. **No limit, no empty fields.** The middle form has just three Lookup fields and nothing else.

THIS WAS A REAL BUG IN THE FIRST DRAFT

The original design put a single `trainer_id` on the subject — so JavaScript could only ever have one trainer. It was caught in review and the column removed.

How to spot it yourself: if you can say "many X have many Y", you need the middle form. Always.

Subform, or separate form?

Creator gives you something a plain database doesn't: **subforms** — child rows living inside a parent record. Powerful, but pick wrong and you rebuild.

USE A SUBFORM WHEN...

The child rows are **only ever seen inside their parent**, and nothing else in the app needs to point at them.

Invoice lines. You always look at them on the invoice. Nothing else references a single line.

USE A SEPARATE FORM WHEN...

Anything else needs to **look up** to it, or you need to report across all of them at once.

Topics. Every class session points at a topic — so Topics must be its own form.

THE TEST THAT DECIDES IT

"Does anything else need a Lookup pointing at this?" If yes → separate form. You cannot build a Lookup field that points at a subform row.

Only two of our 28 become subforms

SUBFORM 1

Billing line items, inside Billing

Each line is one class taught: hours, rate, amount. You only ever read them on the invoice, and nothing points at an individual line.

Bonus: the invoice total is a subform sum — Creator calculates it for you.

SUBFORM 2

Attendance, inside Class Session

One register per class. The trainer opens the session and marks all 30 students inline — **the attendance screen basically builds itself.**

THE CATCH ON ATTENDANCE

Counting across sessions is harder

Certification needs each student's percentage across *every* session. Reading that out of subforms every time is slow.

Fix: a scheduled Deluge job that writes each student's current attendance % onto their enrollment record overnight.

EVERYTHING ELSE

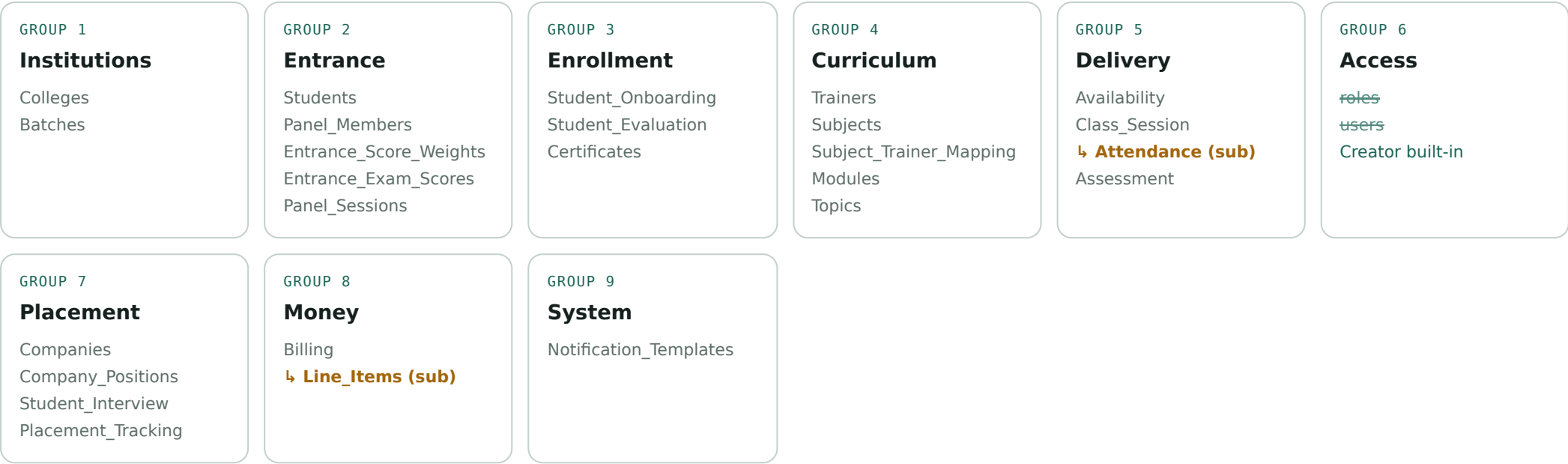
Separate forms

Subjects, Modules, Topics, Availability, Class Session, Students, Batches... all have something pointing at them.

College contacts and panel scores stay as plain fields — capped at 3 by business rule, so a subform buys nothing.

~24 forms, in nine groups

Nobody memorises 24 forms. People remember nine groups, then look up the detail. **Amber = subform. Struck through = Creator gives it free.**



Curriculum nests three deep: **Subject → Module → Topic**. "JavaScript" → "JS Syntax" (12 hrs) → "JS Statements" (1.5 hrs). A topic is what one class actually teaches.

In Creator, *you* are the rulebook.

A full database can refuse bad data at the storage level — it physically cannot be saved. **Creator can't do that.** Which means every rule in the spec is now your responsibility, in Deluge.

WHAT THE SPEC ASSUMES

"The database blocks it." Rules like *weights must total 1* or *one attendance mark per student* are written as guarantees the storage layer enforces.

WHAT YOU'LL ACTUALLY DO

Put each rule in the form's **On Validate** script. It runs before the record saves and can refuse with a message. **That is your constraint.**

BE HONEST ABOUT THE GAP

Deluge validation runs when a **form** is submitted. It does **not** run on a bulk CSV import, and it may not fire on some API writes. So: import carefully, validate imported data separately, and never assume a rule held just because you wrote it once.

Four places a script can run — pick the right one

USE THIS FOR RULES

On Validate

Runs **before** the record is saved. Can stop the save and show a message. Every rule from the spec belongs here.

USE THIS FOR CONSEQUENCES

On Success

Runs **after** a successful save. Send the email, update the related record, recalculate a total. Too late to refuse anything.

USE THIS FOR HELP

On User Input

Runs as someone types. Live feedback — show the weight total as they fill it in, before they hit submit.

USE THIS FOR THE SLOW WORK

Scheduled

Runs nightly. Recalculate attendance percentages, flag sessions with missing registers, prepare draft invoices.

```
// On Validate – Entrance Score Weights
total = input.university_weight + input.attendance_weight + input.apptitude_weight
      + input.hod_weight + input.panel_1 + input.panel_2 + input.panel_3;

if (total != 1.0)
{
  cancel "Weights must add up to exactly 1. Right now they total " + total;
}
```

`cancel` is the word that makes it a rule. Without it you've written a suggestion.

Each one protects something expensive

ENTRANCE SCORING

Weights must total exactly 1

If they sum to 0.95, every candidate is quietly under-scored and the rankings are wrong. Nobody notices — the numbers just look slightly low.

Protects: fair selection.

SCORING AGAIN

No total until all 3 panelists have scored

Panelists interview on different days, so scores arrive piecemeal. Saving a half-scored candidate is fine. **Ranking one is not.**

Protects: fair selection.

ATTENDANCE

One mark per student per class

Otherwise a student is marked present twice and absent once for the same session. Which is true? Nobody can say.

Protects: honest records — and next slide, this one is genuinely tricky in Creator.

BILLING

A class can be billed only once

Generate June's invoice twice and you pay for the same class twice.

Protects: real money.

And the fifth: no certificate below 80% attendance. That's a promise to employers, so it can't depend on someone remembering to check.

How to stop the same thing being recorded twice

Creator can mark a **single field** unique. But "one mark per student *per session*" is two fields together — and Creator has no built-in way to say that.

THE OBVIOUS WAY, AND ITS FLAW

In **On Validate**, search for an existing record with the same session and student. If found, `cancel`.

Works nearly always. But if two saves land at the same instant, both searches come back empty and **both save**. Rare — not impossible.

THE STURDIER WAY

Add a hidden Single Line field marked **Unique**. Fill it in script with the two IDs joined together.

Now Creator itself refuses the duplicate, because it's one field again. **The uniqueness is enforced by storage, not by your search.**

```
// On Validate – build the key that cannot repeat
input.unique_key = input.Class_Session + "-" + input.Student;
```

Use the same trick for billing: a unique key on the class session in the invoice line, so one class can never appear on two invoices. Check it works in your Creator version before relying on it — then reuse it everywhere the spec says "unique when active".

Copy the rate onto the invoice. Don't look it up later.

Priya teaches at ₹800/hour in June. In July she gets a raise to ₹1,000/hour.

What should June's invoice say?

IF THE LINE READS HER RATE LIVE FROM TRAINERS

June's invoice was 8 hrs × ₹800

₹6,400 → ₹8,000

The moment you save her raise, **every past invoice silently rewrites itself**. Paid, closed months change value. Your books stop matching your bank.

IF THE LINE STORES ITS OWN COPY

The line saves `rate_applied = 800`

₹6,400 → ₹6,400

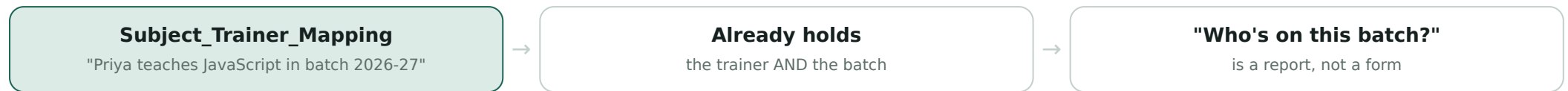
June stays ₹6,400 forever. July uses ₹1,000. **History freezes; the future moves on.**

In Creator this is a real trap, because a Lookup makes it so easy to just show the trainer's current rate on the invoice. Don't. Copy the number into a Currency field on the line when the invoice is created, and read that from then on.

The general rule: when a number is part of a finished transaction — a price, a rate, a tax percentage — store a copy on that transaction. Look values up live only when you *want* them to change.

Everything else: work it out, don't store it twice

We nearly shipped a form listing which trainers work on which batch. We deleted it — the mapping form already knows.



The danger of keeping both: remove Priya from the batch list but leave her subject records, and the app now holds two contradictory answers to one question. Whichever report you happen to open decides what you believe. **Nothing errors — it just quietly disagrees with itself.**

SO WHICH IS IT — COPY, OR CALCULATE?

Copy when the value must never change again (money on a finished invoice). **Calculate** when it must always be current (who teaches this batch). The one grey area is our overnight attendance percentage — a stored copy purely because recalculating it constantly would be slow. **Know which you're doing and why.**

Reports and Pages are your 23 screens

Every wireframe is annotated with the exact fields it reads and writes — so building a screen is mostly translation, not invention.

CREATOR DOES THE WORK

List & Grid reports

Colleges, companies, positions, curriculum, candidates. Generate the report, hide the fields you don't want, done. **Roughly half the screens.**

YOU DO THE WORK

Pages + custom forms

The dashboard, entrance scoring, the timetable mapper, invoice generation. Pages with panels, and forms driven by Deluge.

FREE ON MOBILE

The trainer's five

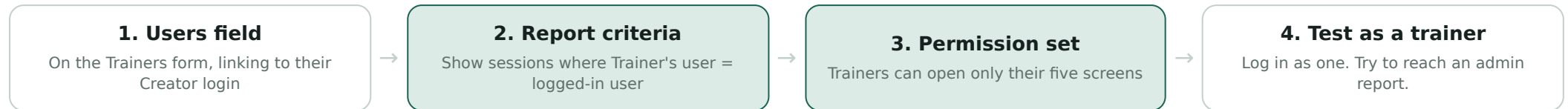
Built as ordinary forms and reports, opened in the Creator mobile app. **Design for a thumb:** big dropdowns, few fields, no typing where a tap will do.

ONE PRINCIPLE WE USED ON EVERY SCREEN

If a rule forbids it, don't offer it. The "Issue certificate" button is hidden below 80%. The timetable only lists trainers who actually teach that subject in that batch. **You can't make the mistake, so you never see the error.** In Creator that means filtered lookups and conditional buttons — not just a `cancel` message after the fact.

How a trainer only ever sees their own work

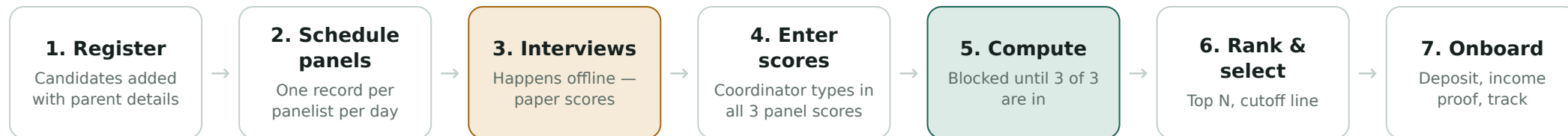
This is the single most important permission in the app — and it's easy to get wrong in a way nobody notices until a trainer sees someone else's pay.



That Users field is what the spec calls `trainers.user_id` — it exists precisely so a login can be tied back to a trainer record. It's the hinge the whole permission model hangs on.

Step 4 is not optional. Permission bugs are invisible to the person who wrote them — you always test as yourself, and you're an admin. Make a test trainer login on day one and keep using it.

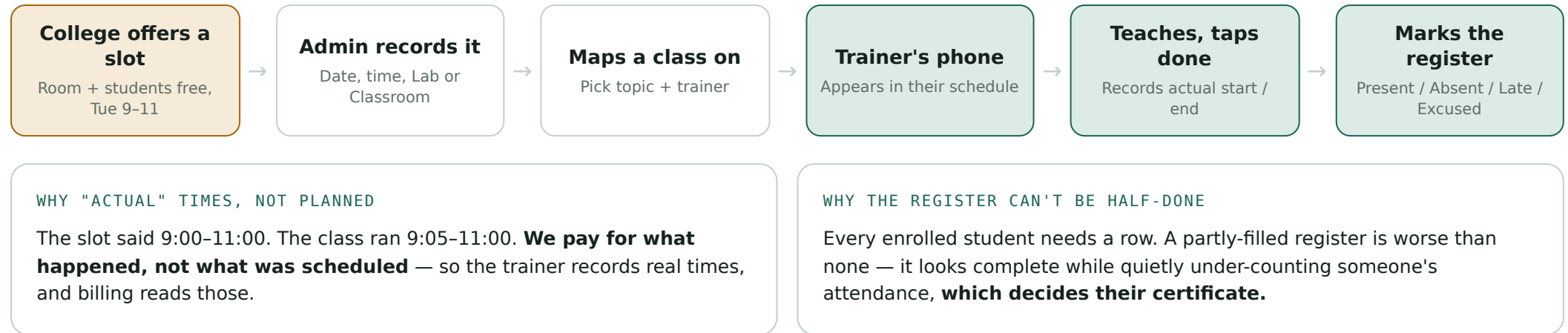
Entrance: from applicant to enrolled student



Amber = happens in the real world, not in the app. Worth marking, because that's where data gets delayed or lost.

Panel members never log in. They score on paper; a coordinator types it all in. That was a deliberate decision — it means no panelist accounts, no training them on the app, and one person responsible for the numbers.

Running a class, and marking who came



Creator tip: pre-fill the attendance subform with the batch's enrolled students when the session opens, so the trainer only changes the exceptions. Everyone defaults to Present; they tap the three who aren't.

How a taught hour becomes a paid invoice



Every rupee traces back to a specific class on a specific day. If Priya questions her invoice, you show her the exact sessions on it. That's why line items exist at all — the original design had only a total, which is impossible to defend or debug.

TWO KINDS OF PAYEE, ONE SET OF FORMS

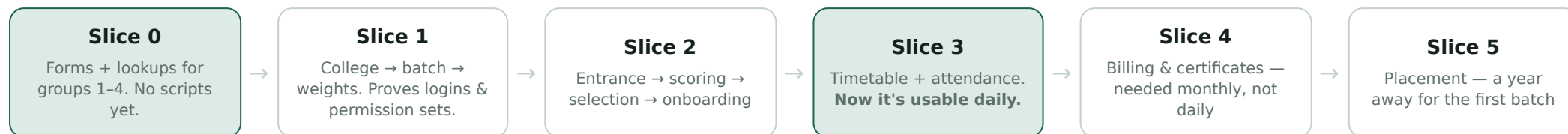
Trainers bill for classes; panel members bill for interview sittings. Same Billing form. A rule guarantees each invoice has **exactly one** payee — never zero, never both.

ATTENDANCE NEVER AFFECTS PAY

A trainer is paid for teaching whether 30 students came or 4. **The attendance chain and the money chain stay completely separate.**

Thin slices, not layers

The instinct is to build all 24 forms, then all the reports, then all the scripts. Don't. **Build one complete journey at a time** — each slice is something you can show a coordinator and get corrections on.



Why this order: it follows the student's journey, so every slice is real and demoable. The first invoice isn't due until a month after teaching starts, and placement isn't needed until the first batch graduates. **Get the daily screens right first.**

One warning: get the lookups and the two subforms right in Slice 0. Changing a field into a subform later, after there's data in it, is painful.

Three decisions needed before you build the forms

Everything else is settled. These three change the **structure**, and structure is cheapest to change before there's data in it.

QUESTION 4

Does the Zoho vs non-Zoho track change what's taught? Nothing currently connects a student's track to a set of subjects. If the tracks study different things, you need another lookup — and the timetable must filter by it.

QUESTION 7

Does the app generate certificate PDFs, or just record that one was issued? We store a template link but have nowhere to put the finished file. Creator can generate PDFs — if we use that, we need a field for the output.

QUESTION 10

What happens to the deposit if a student drops out? Refunds are defined for students who complete. Dropouts aren't — and that's exactly when someone asks for their money back.

There are ten open questions in total; the other seven affect screens rather than structure, so they can be answered while you build.

Six things to remember

1 · Nouns become forms

Including events — "a class happened", "someone was present", "an invoice was raised".

2 · The many side holds the lookup

And when both sides are "many", build a mapping form in the middle.

3 · Subform only if nothing points at it

If another form needs a Lookup to it, it must be its own form.

4 · You are the constraint

Every rule lives in **On Validate**, ending in `cancel`. Creator won't catch it for you.

5 · Freeze money, calculate the rest

Copy rates onto invoice lines. Everything else, work out from one source of truth.

6 · Never delete — deactivate

Set `active` to false. History has to stay standing, or last year's invoices stop making sense.

Where everything lives: APP-Data-Structure-Detailed.md has all 28 tables with every field explained. APP-Wireframes-Storyboards.html has the 23 screens and 10 flows, each annotated with the fields it touches.

Questions now — and **if something in the design looks wrong to you, say so.**

Six real bugs were caught by exactly that kind of second look, including the one-trainer-per-subject mistake on slide 9.